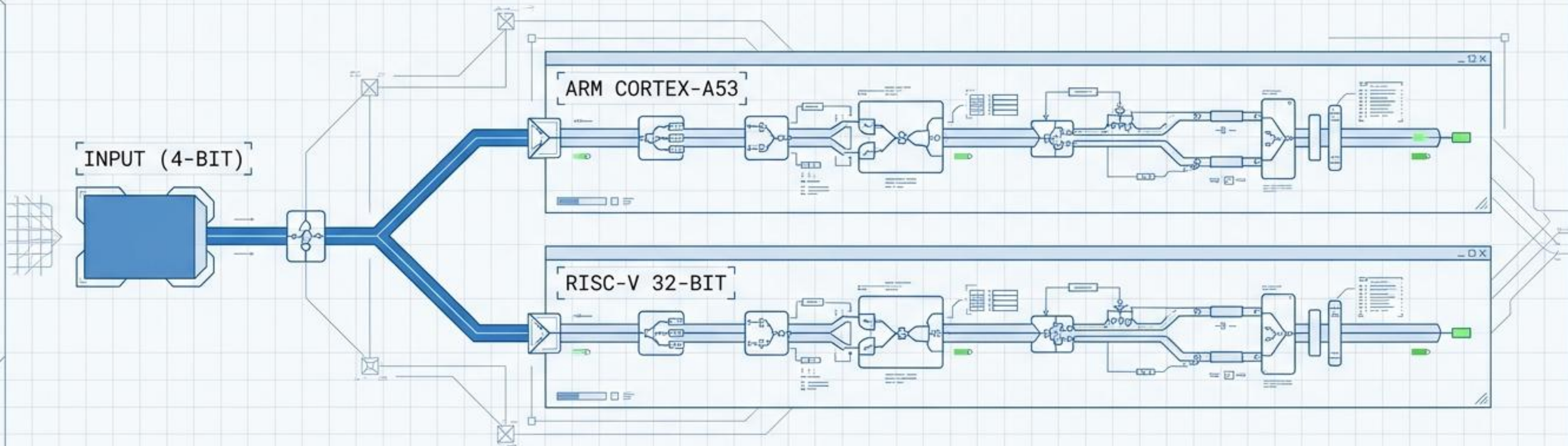


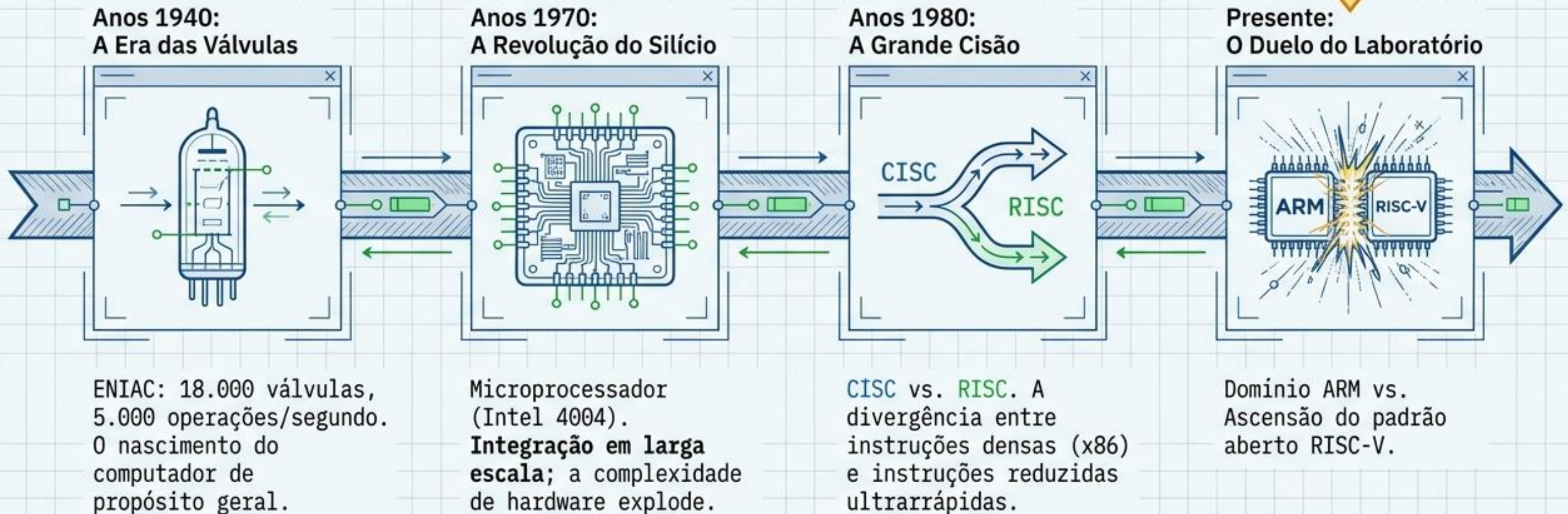
Arquiteturas em Duelo

Desafio da Calculadora Binária: Implementação ARM (Raspberry Pi 3) vs. RISC-V (ESP32).



A Evolução Arquitetural até a Bancada

A Aula 08 testa na prática o legado arquitetural do design RISC.

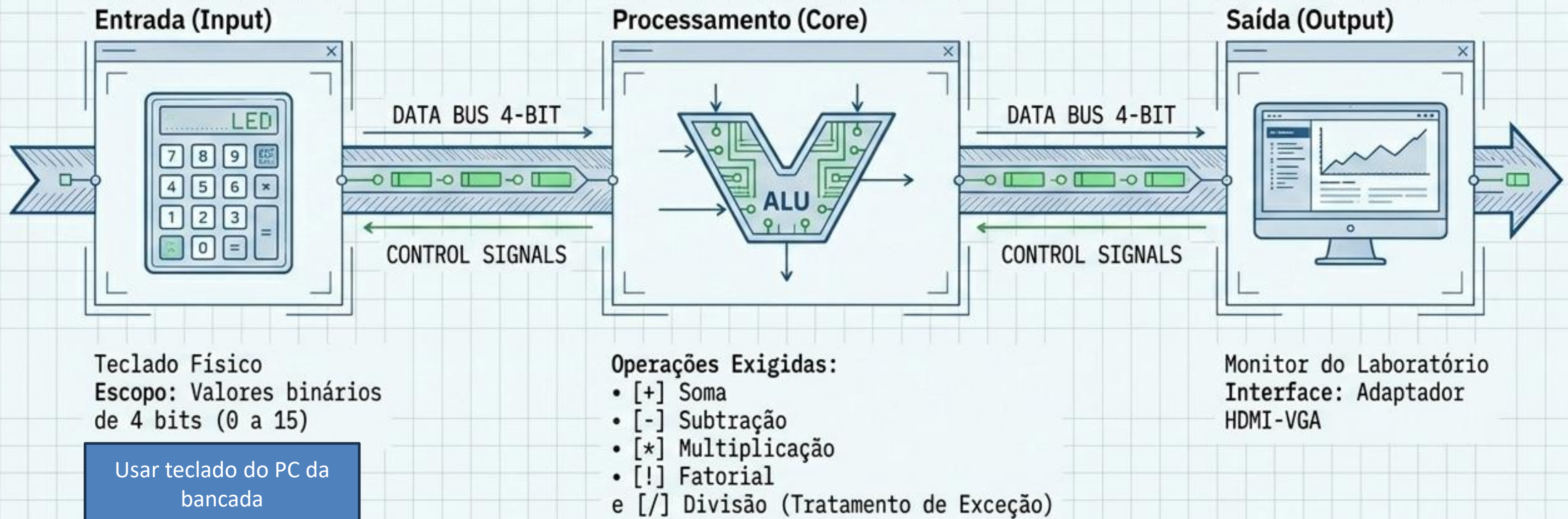


Escopo Prático: Esquemático de Requisitos

Aviso de Engenharia

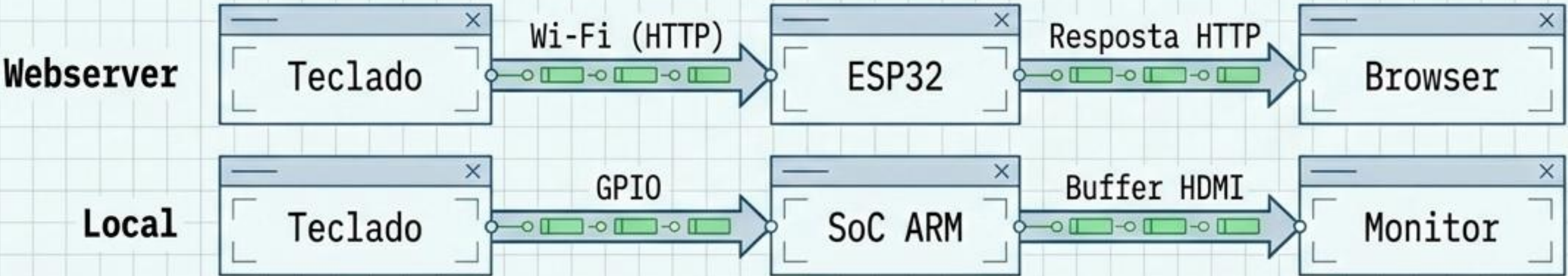
PLATAFORMAS ALVO:

1. Raspberry Pi 3 (ARM Cortex-A53)
2. ESP32 (RISC-V 32-bit)

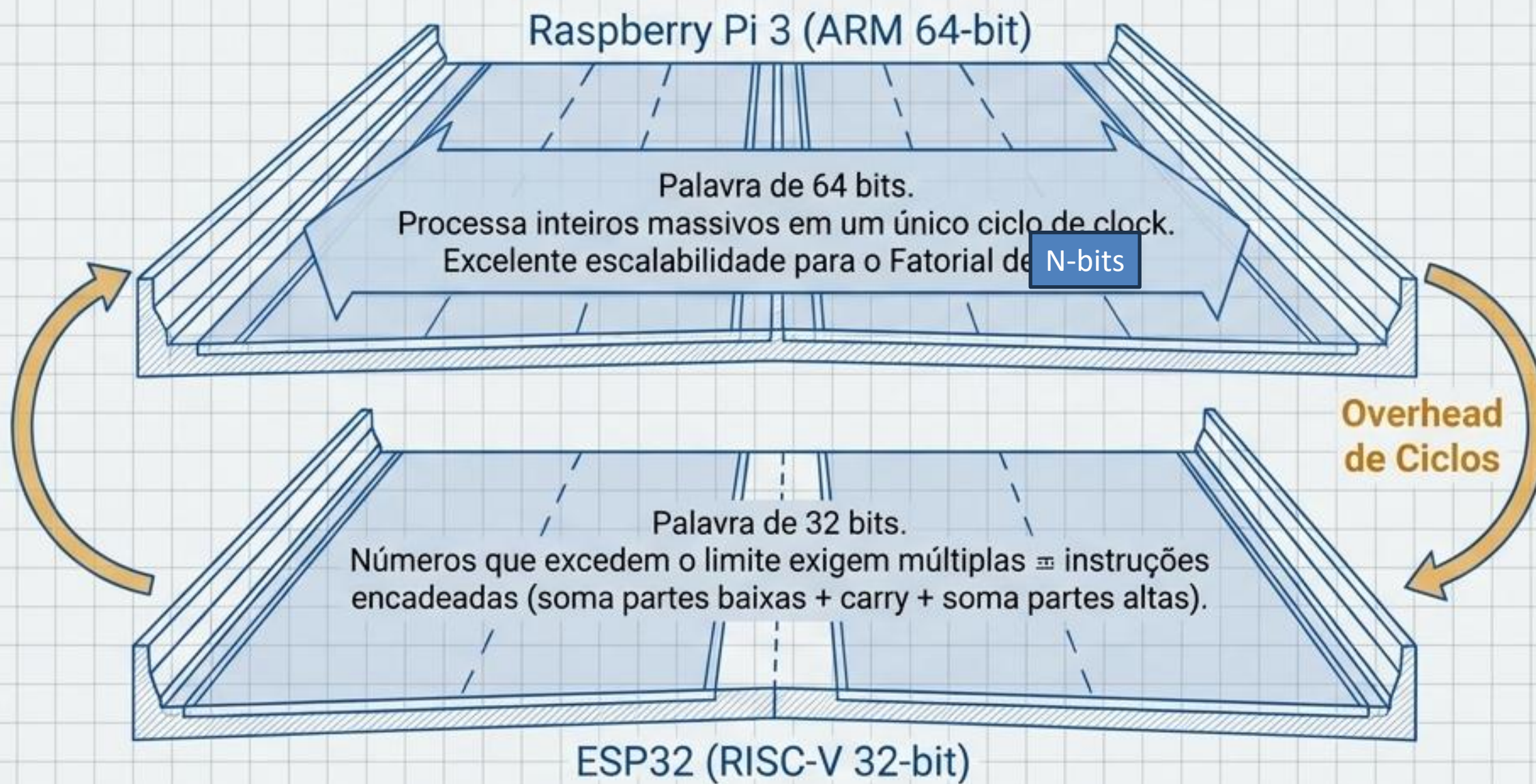


Matriz de Diagnóstico: ARM vs. RISC-V

	ESP32 (RISC-V 32-bit)	Raspberry Pi 3 (ARM 64-bit)
Foco Arquitetural	Baixo consumo, IoT, rotinas determinísticas.	Processamento de uso geral, multitarefa.
Paradigma de I/O	Webserver via rede TCP/IP (WiFi) gera latência de comunicação.	Acesso local direto via GPIO e barramento de vídeo (HDMI-VGA).
Camada de Sistema	Baremetal / RTOS. Controle direto de interrupções de hardware. Utilizado apenas Baremetal	Sistema Operacional Linux. Agendamento de processos e abstração pesada.



Escalabilidade Computacional: Processando Números Grandes



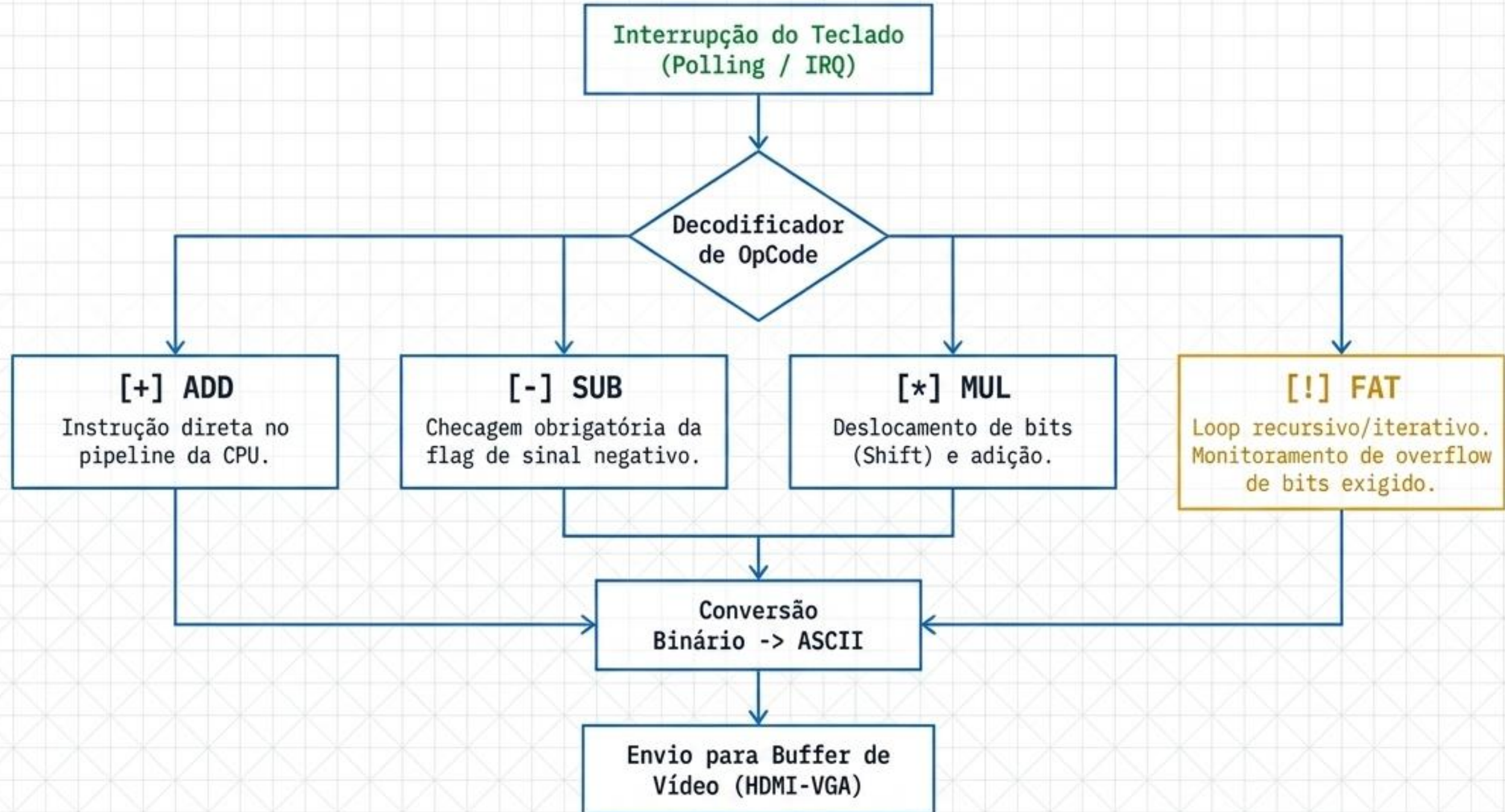
Fazer a comparação para n-bits
(de 4 bits até 64 bits por exemplo)

Metodologia: Planejamento Pré-Código

Requisito (RF/RNF)	Teste Planejado	Resultado Esperado	Status / Evidência
RF01: Entrada 4 bits	Digitar sequência binária "1111".	Leitura do valor decimal 15 no registrador alvo.	
RF02: Saída HDMI	Injeção de string arbitrária na porta de vídeo local.	Visualização imediata no monitor da bancada.	
RNF01: Estabilidade de Erro	Inserir operação inválida (ex: A/0).	Sistema exibe aviso, não sofre kernel panic e aguarda novo input.	

> AVISO METODOLÓGICO: A execução cega gera retrabalho. Especifique os casos de teste rigorosamente antes de escrever a primeira linha de Assembly.

Arquitetura Lógica: Fluxo da Unidade Lógica Aritmética (ULA)



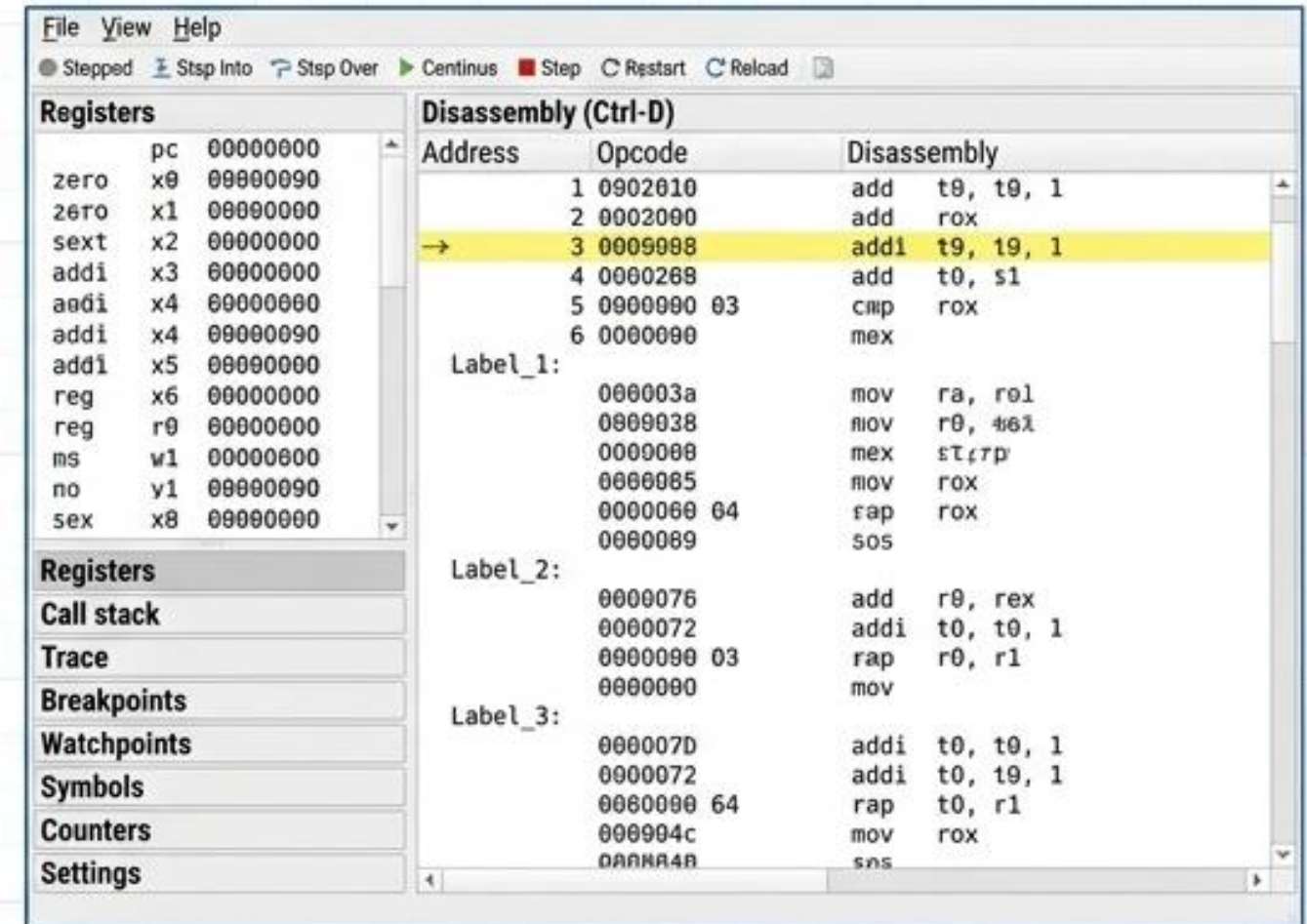
A Magia da Simulação: Geração Sintética e Validação

Geração de Sintaxe (IA Generativa)

```
PROMPT > Gere uma rotina em Assembly ARM  
para multiplicar dois números de 4 bits.
```

```
MUL R2, R0, R1  
CMP R2, #15  
BGT overflow_handler
```

Validação Lógica (CPUlator)



The screenshot shows the CPUlator application interface. On the left, a 'Registers' panel lists various registers (pc, x0-x8, r0-r1, w1, y1, x8) and their current values. On the right, a 'Disassembly (Ctrl-D)' panel shows a list of assembly instructions with their addresses and opcodes. The instruction at address 00000003 is highlighted in yellow.

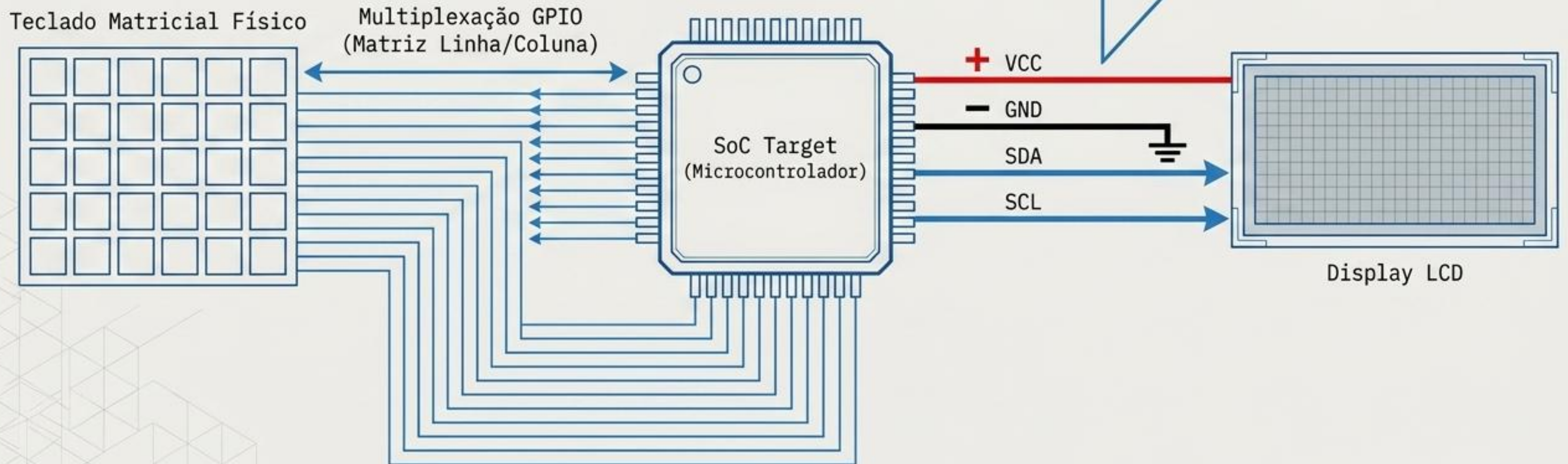
Address	Opcode	Disassembly
1	0902010	add t0, t0, 1
2	0002000	add rox
→ 3	0009008	addi t9, t9, 1
4	0002008	add t0, s1
5	0000000 03	cmp rox
6	0000000	mex
Label_1:		
	000003a	mov ra, r01
	0000038	mov r0, 461
	0000000	mex st, r0
	0000005	mov rox
	0000000 64	rap rox
	0000009	sos
Label_2:		
	0000076	add r0, rex
	0000072	addi t0, t0, 1
	0000000 03	rap r0, r1
	0000000	mov
Label_3:		
	0000070	addi t0, t0, 1
	0000072	addi t0, t9, 1
	0000000 64	rap t0, r1
	000004c	mov rox
	0000040	sns

A IA escreve a sintaxe. O engenheiro valida a lógica.

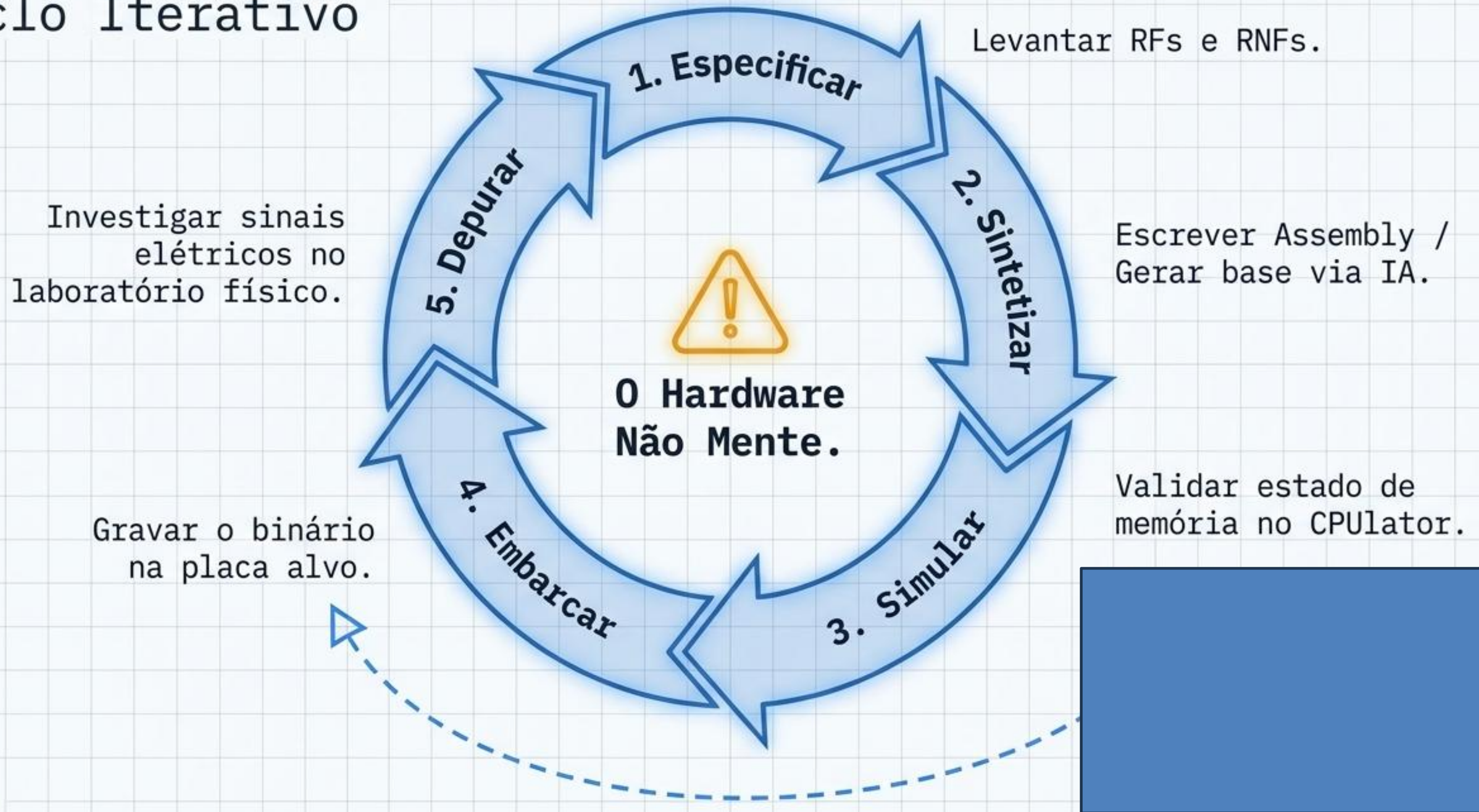
0 Desafio Standalone: Desacoplando do PC



Barramento I2C: Reduz drasticamente a contagem de pinos de dados (de 8+ paralelos para apenas 2). Exige escrita de drivers de baixo nível no código para envio de bytes de controle.



0 Método Experimental: Ciclo Iterativo



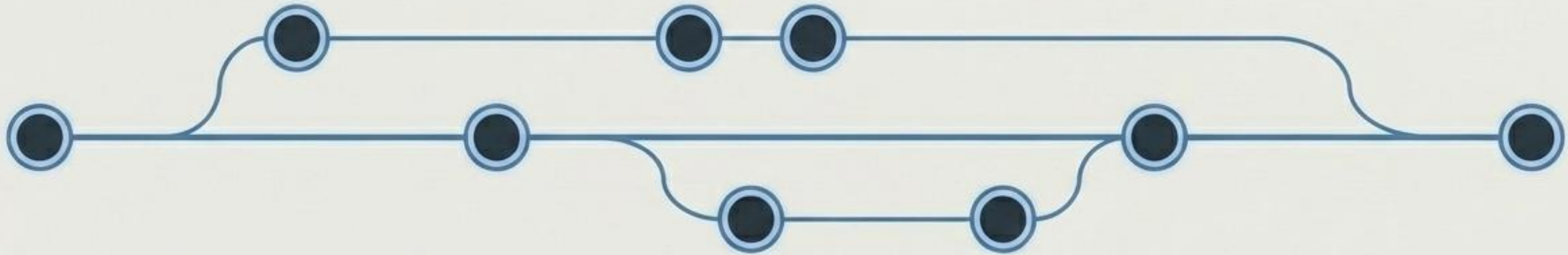
Arquitetura da Entrega: O Mapa do Relatório



Checklist de Restrições	
Formato Obrigatório: Documento PDF.	✓
Padrão de Qualidade: Citações e formato ABNT rigoroso.	✓
Restrição Tipográfica: Fonte limpa (equivalente a Arial 12).	✓
Limitação de Escopo: Máximo absoluto de 10 páginas.	✓

Documentação não é burocracia; é a prova da replicabilidade da sua engenharia.

Transparência e Versionamento (GitHub)



Acesso Público

Link do repositório deve estar funcional e explícito na primeira página do relatório PDF.

Versionamento Contínuo

O histórico deve refletir a evolução desde a Semana 1. Um único commit final será penalizado.

Documentação de Build

README.md detalhado contendo instruções precisas para compilação e deploy na placa física.

Perspectiva: Até mesmo o código crítico da AGC da Apollo 11 reside hoje em repositórios públicos. Preservação de código é o padrão ouro da engenharia.